

Teoría de la computación y lenguajes

Trabajo Practico 2

Lucas Javier Martinez
1° Cuatrimestre de 2026

Índice de contenidos

Índice de contenidos	1
Enunciado	2
Respuesta	3

Enunciado

Generar un compilador utilizando Antlr4 capaz de procesar el código de prueba incluido en la entrega.

Se pide:

- **Generar la gramática necesaria**
- **Generar el árbol asociado**
- **Realizar pruebas del código correcto**
- **Realizar pruebas de generación de errores a partir de un código erróneo.**

Entregable:

- **Archivos de código del compilador, incluyendo la gramática .g4**
- **Árbol generado**
- **Informe de resultados y conclusiones.**

Respuesta

En el presente trabajo, nos centramos en crear una solución a un enunciado relacionado al procesamiento de residuos, el mismo consistió de un código similar a python, pero con modificaciones en la sintaxis como por ejemplo la incorporación de llaves para hacer más sencilla la tarea de creación de una gramática, el código en cuestión es el siguiente:

```
fn controlar_residuos(nivel_peligrosidad, cantidad, destino) {  
    if (cantidad > 450) {  
        print("El residuo ha sido rechazado porque pesa mas de lo permitido  
(450kg).");  
        return "Rechazado";  
    } else {  
        print("El residuo con peligrosidad ", nivel_peligrosidad, " de ", cantidad,  
"kg y destino ", destino, " ha sido procesado.");  
        return "Procesado";  
    }  
}
```

Dicho esto, se procedió a crear la gramática para poder compilar este código, que terminó siendo la siguiente

```
grammar MiniPython;  
  
// Reglas sintácticas (Parser)  
program      : functionDef* EOF ;  
  
functionDef: 'fn' ID '(' paramList? ')' block ;  
  
paramList  : ID (',' ID)* ;
```

```
block      : '{' statement* '}' ;
```

```
statement  : ifStat  
            | returnStat ';' ;  
            | printStat ';' ;  
            | expr ';' ;  
            ;
```

```
ifStat     : 'if' '(' expr ')' block ('else' block)? ;
```

```
returnStat : 'return' expr ;
```

```
printStat  : 'print' '(' expr (',' expr)* ')' ;
```

```
expr       : ID  
            | INT  
            | STRING  
            | expr simbolo expr  
            ;
```

```
simbolo    : '>' | '<' | '==' | '+' ;
```

```
// Reglas léxicas (Lexer)
```

```
ID         : [a-zA-Z_][a-zA-Z0-9_]* ;
```

```
INT        : [0-9]+ ;
```

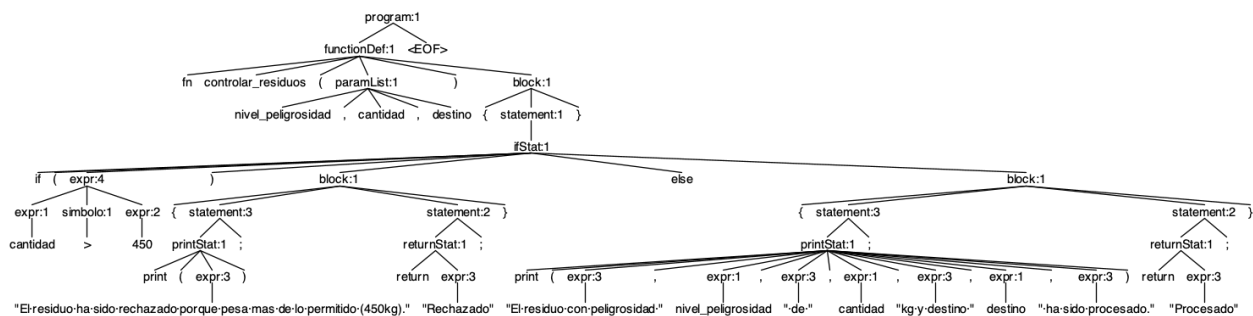
```
STRING      : ''' (~[\r\n"])* ''' ;
```

```
WS          : [ \t\r\n]+ -> skip ; // Ignora espacios y saltos de línea
```

Lo primero que podemos resaltar, es que se optó por una gramática que solo acepte funciones, que adentro tendrá sentencias.

Algunos puntos a considerar de esta gramática, es que debimos hacer que la lista de parámetros sea opcional, ya que podemos tener una función que no tenga parámetros tranquilamente. Luego también es bueno remarcar tenemos una similitud entre `paramList` y la sentencia entre parentesis en la definición del `print`, podríamos pensar que son equivalentes y se podrían agrupar en una sola sentencia, pero esto no es correcto, ya que no debemos permitir cualquier expresión en la lista de parámetros, sino solamente variables (es decir IDs)

A continuación vemos el árbol generado mediante `antlr4`



Luego, propusimos dos ejemplos de código incorrecto que dieron problemas de sintaxis en la compilación; el primero de ellos simplemente no posee la llave de cierre:

```
fn controlar_residuos(nivel_peligrosidad, cantidad, destino) {
    if (cantidad > 450) {
        print("El residuo ha sido rechazado porque pesa mas de lo permitido
(450kg).");
        return "Rechazado";
    } else {
```

```
print("El residuo con peligrosidad ", nivel_peligrosidad, " de ", cantidad,
"kg y destino ", destino, " ha sido procesado.");

return "Procesado";

}
```

Por otro lado, el segundo ejemplo es un código python que dará varios errores por tener caracteres no definidos:

```
def controlar_residuos(nivel_peligrosidad, cantidad, destino):

    return 1
```

Como conclusión, podemos ver que una sintaxis con funciones y llaves es mucho más sencilla que la tradicional de python, por otro lado, es algo interesante que antlr no nos priva de ver el árbol frente a pequeñas omisiones de sintaxis, sino que nos lo muestra y solamente nos alerta del error, esto hace que sea más fácil debuggear y no se muestre un solo error, haciendo fácil que hagamos todas las correcciones necesarias juntas.